< Programme Grande École — 3e année />

EPIKODI_

Un media center extensible, piloté depuis le navigateur d'un smartphone.

Sujet de projet professionnel : alternative au part-time en entreprise.

01 Le projet

02 Le produit attendu

03 Exigences de qualité

04 Organisation et livrables

Format : groupes de 2 étudiants, le jeudi et le vendredi, en présentiel

Période : du 22 octobre 2026 à fin février 2027

Année : 2026–2027 · Epitech Paris

Auteur : Joffrey RIELA, directeur pédagogique PGE

{Epitech}

01

LE PROJET_

A. CONTEXTE

Vous développez **votre propre media center**, inspiré de Kodi (le media center libre bien connu) : une application de salon qui gère une médiathèque, enrichit automatiquement les médias de leurs informations (affiches, résumés, casting), les lit, et se pilote depuis le **navigateur d'un smartphone**, qui sert de télécommande sans aucune installation.

> `< Principe />` Inutile de réécrire un lecteur vidéo : appuyez-vous sur une bibliothèque existante (libmpv, VLC, GStreamer…). Le cœur du sujet est **tout ce qui l'entoure** : médiathèque, interface de salon, extensions et pilotage à distance.

B. CADRE

- **Autonomie totale :** aucun matériel, aucune codebase et aucun apport technique ne sont fournis.
- **Stack libre**, à justifier dans la documentation technique.
- **Usage de l'IA autorisé**, encadré et déclaré (voir bloc 03-D).
- **Projet pédagogique :** le produit n'a pas vocation à être diffusé en dehors du projet.

C. RÈGLES D'OR

- **Le produit a sa propre marque.** « EpiKodi » est le nom du sujet, pas du produit. Aucune reprise du nom, du logo ou de l'interface de Kodi, ni référence à Epitech.
- **Contenus légaux uniquement :** des médias libres de droits ou sous licence libre (films de la Blender Foundation, domaine public, musique Creative Commons…).
- **Aucune extension de piratage.** Aucune extension ne donne accès à des contenus diffusés illégalement.
- **API externes dans les règles.** TheMovieDB, MusicBrainz… sont utilisés selon leurs conditions, avec l'attribution demandée.

D. POINTS D'ATTENTION PARTICULIERS

Au-delà des fonctionnalités, trois aspects feront l'objet d'une attention particulière (détails dans le bloc 03) :

- **Accessibilité :** une interface de salon et une télécommande utilisables par tous, sur les critères essentiels des WCAG 2.2 (niveau AA).
- **Qualité du dépôt :** un dépôt propre, des bonnes pratiques et une installation documentée pas à pas.
- **Documentation :** utilisateur, technique, et guide de création d'extensions.

02

LE PRODUIT ATTENDU_

A. IDENTITÉ DU PRODUIT

- Nom, logo et charte graphique **originaux**.
- Un **thème par défaut** soigné, conçu pour un écran de télévision.
- Une mini charte graphique (1 à 2 pages) est livrée.

B. APPLICATION DE SALON

- Application de bureau **sous Linux**, lancée en plein écran.
- **Moteur de lecture obligatoire** : la lecture passe par une vraie bibliothèque de lecture (libmpv, VLC, GStreamer…), jamais par le lecteur intégré d'un navigateur, y compris dans une application construite sur un moteur web (Electron, Tauri…).
- **Interface « 10 pieds » :** lisible à 3 mètres d'un téléviseur, entièrement navigable avec les flèches, Entrée et Retour, sans souris, avec un focus toujours très visible.
- Écran d'accueil : reprise de lecture, derniers ajouts, accès aux films, séries, musique et extensions.

C. MÉDIATHÈQUE

Sources locales et réseau

Dans un foyer, les médias sont rarement stockés sur la machine branchée à la télévision : ils se trouvent souvent sur un disque réseau (NAS), un autre ordinateur ou la box internet. Le media center doit donc savoir lire des fichiers **à distance**.

- **Dossiers locaux** de la machine, et **partages SMB** (le partage de dossiers proposé par la plupart des NAS et des box).
- L'utilisateur ajoute une source réseau depuis l'interface (adresse, identifiants si besoin).
- Un partage lent, inaccessible ou supprimé est géré proprement : la médiathèque reste consultable et l'utilisateur est informé clairement.
- Les identifiants des partages sont stockés de façon sécurisée, jamais en clair.
- Pour la démonstration, le partage réseau peut être simulé dans un conteneur (par exemple un serveur Samba sous Docker).

Reconnaissance et enrichissement

- Analyse des sources (scan) initiale puis **incrémentale** : seuls les fichiers ajoutés, modifiés ou supprimés sont traités.
- Reconnaissance des noms de fichiers : film et année, série, saison et épisode (`Serie.S01E02.mkv`, `1x02`…), artiste et album à partir des tags audio.
- Enrichissement automatique via des API publiques : affiche, fond, résumé, casting, genres, durée, pochettes.

- **Correction manuelle :** l'utilisateur peut corriger une reconnaissance erronée et choisir parmi plusieurs résultats.
- Cache local des informations et des images : la médiathèque reste consultable hors connexion.

Organisation

- Vues par films, séries (saisons, épisodes) et albums.
- Recherche, favoris, statut vu / non vu.

D. LECTURE

- **Vidéo :** formats courants (MP4, MKV, WebM), choix de la piste audio, sous-titres SRT.
- **Musique :** lecture par album, file d'attente.
- **Reprise de lecture** là où on s'était arrêté.
- Enchaînement automatique de l'épisode suivant.

E. EXTENSIONS ET THÈME

Une extension est un module indépendant qui ajoute une fonctionnalité au media center sans modifier son code, comme les extensions d'un navigateur ou les add-ons de Kodi. Le sujet demande de construire **le système qui les accueille**, pour que n'importe quel développeur puisse en créer de nouvelles.

Le système d'extensions

- Une extension est un paquet contenant un **manifeste** (nom, version, auteur, description, type, permissions demandées) et son code.
- Le produit fournit une API pour les extensions : au minimum **ajouter une source de médias** et **afficher un élément sur l'écran d'accueil**.
- Depuis l'interface, l'utilisateur installe une extension depuis un fichier, la désactive ou la désinstalle, **sans redémarrer ni recompiler** le produit.
- **Système conçu par le groupe :** réutiliser le mécanisme d'extensions d'un navigateur ou d'un framework ne répond pas à l'exigence.

Permissions et isolation

- Chaque extension déclare dans son manifeste ce dont elle a besoin (accès à internet, lecture de la médiathèque…). L'utilisateur voit et accepte ces permissions à l'installation, et le produit refuse tout le reste.
- Une extension tourne **dans un processus séparé** : si elle plante ou se bloque, elle est arrêtée sans faire planter le media center, et l'utilisateur en est informé.

Extensions et thème à fournir

- **Deux extensions** : une source de contenus (par exemple un catalogue de films du domaine public) et un widget pour l'écran d'accueil (météo, minuteur de mise en veille…).
- Le thème par défaut est décrit dans un format documenté, indépendant du code de l'interface.

F. PILOTAGE DEPUIS LE TÉLÉPHONE

> `< Pas d'application à installer />` La télécommande est une **page web servie par le media center**. Le téléphone l'ouvre dans son navigateur en scannant un QR code affiché à l'écran. Aucune application native n'est attendue.

API de contrôle

- Le media center expose une API sur le réseau local (HTTP / WebSocket) : navigation, lecture, volume, médiathèque, état en temps réel.
- **Appairage sécurisé :** un téléphone doit être autorisé par un code affiché à l'écran avant de pouvoir piloter.

Télécommande

- Pavé directionnel, Entrée et Retour, lecture / pause, avance, volume.
- **Navigation dans la médiathèque depuis le téléphone :** parcourir les films, séries et albums, rechercher, et lancer un média sur l'écran.
- Affichage en temps réel de ce qui est en cours de lecture, avec la progression.
- Compatible au minimum avec Safari (iOS) et Chrome (Android).

Le téléphone et le media center doivent être sur le même réseau. Si le wifi utilisé isole les appareils entre eux, prévoyez un partage de connexion pour vos démonstrations.

G. MÉDIATHÈQUE DE DÉMONSTRATION

- Une médiathèque légale prête à l'emploi : au moins **5 films**, **une série** de 3 épisodes minimum et **20 morceaux**.
- Elle inclut volontairement des **noms de fichiers difficiles** (mal formatés, ambigus) pour démontrer la reconnaissance et la correction manuelle.
- Un script la télécharge et l'installe automatiquement, avec la licence de chaque média.

H. BONUS

Valorisés, non obligatoires : second système (Windows ou macOS), profils multiples et contrôle parental, découverte automatique des partages et du media center (mDNS), podcasts, listes de lecture et filtres avancés, réglage de la taille et du décalage des sous-titres, second thème interchangeable à chaud, file d'attente et choix des sous-titres depuis le téléphone, « envoyer vers la télé », release GitHub automatique.

03

EXIGENCES DE QUALITÉ_

Ces exigences sont **obligatoires**. Elles comptent autant que les fonctionnalités.

A. ACCESSIBILITÉ

Le produit doit être utilisable par tous, y compris par des personnes en situation de handicap. Le référentiel est le **niveau AA des WCAG 2.2**, transposé en France par le **RGAA**, appliqué à l'interface de salon et à la télécommande, sur les critères suivants.

Exigences minimales

- **Lecteurs d'écran :** compatibilité avec le lecteur d'écran du système (Orca) ou synthèse vocale intégrée des éléments sélectionnés sur l'interface de salon. HTML sémantique, titres hiérarchisés et libellés sur tous les contrôles de la télécommande.
- **Contrastes** conformes (4,5:1 pour le texte courant), tailles de texte adaptées à la distance, et information **jamais portée par la seule couleur**.
- **Images :** affiches et pochettes accompagnées d'un texte alternatif (titre du média) sur la télécommande.
- **Formulaires** (ajout d'une source, appairage, correction d'une reconnaissance) : erreurs explicites, liées au champ concerné, avec une suggestion de correction.

Preuves attendues

- Un **audit automatique** : Lighthouse, axe ou WAVE pour la télécommande, et un outil d'inspection (Accerciser, Accessibility Insights…) pour l'interface de salon, avec les corrections apportées.
- Un **test au lecteur d'écran** sur le parcours principal : Orca (ou synthèse vocale intégrée) sur le media center, VoiceOver ou TalkBack sur la télécommande, de la recherche d'un film jusqu'à sa lecture.
- Une **déclaration d'accessibilité** : niveau atteint, limites connues, actions prévues.

B. DÉPÔT, MÉTHODE ET BONNES PRATIQUES

> `< Installation documentée />` Le README décrit pas à pas les prérequis, l'installation, le lancement avec la médiathèque de démo et les tests. Un développeur extérieur au groupe doit pouvoir installer le media center **sans aide**.

Gestion de projet avec GitHub Projects

- Un tableau GitHub Projects lié au dépôt, **configuré et tenu à jour** du début à la fin.
- Les tâches sont des issues, avec un responsable, un label et un statut, découpées en milestones ou en itérations.
- Chaque développement passe par une **pull request** reliée à son issue et relue avant merge. La branche principale est protégée.
- L'historique doit montrer **qui a fait quoi, et quand**.

Fichiers indispensables

- `README.md` : présentation, captures, prérequis, installation, lancement, tests, liens vers la documentation.
- `CONTRIBUTING.md` : conventions et déroulé d'une contribution.
- `LICENSE`, `CHANGELOG.md`, `CODE_OF_CONDUCT.md`.
- Modèles d'issue et de pull request (`.github/`).
- `.env.example` documenté, et aucun secret (clés d'API comprises) dans le dépôt ni dans l'historique.
- `.gitignore` adapté : ni dépendances, ni builds, ni médias.

Bonnes pratiques

- Arborescence claire, sans code mort ni fichiers orphelins.
- Linter et formateur appliqués automatiquement (pre-commit ou CI).
- Convention de commits commune (par exemple Conventional Commits).
- Stratégie de branches documentée.
- Versions taguées aux étapes clés (versionnage sémantique).
- Dépendances à jour et verrouillées (lockfile).

Sécurité et robustesse

- L'API de contrôle n'accepte que les téléphones appairés. Les échanges sont authentifiés.
- Les extensions sont isolées : une extension défaillante ou malveillante ne peut ni faire planter le produit ni sortir de ses permissions.
- Le produit reste utilisable sans connexion internet et quand une API externe ne répond pas.
- **Performances mesurées :** temps de scan d'une médiathèque de 1 000 fichiers.

Tests

- Des tests unitaires ou fonctionnels couvrent au minimum la reconnaissance des noms de fichiers (avec un jeu de cas difficiles), le scan incrémental, la reprise de lecture, le cycle de vie et les permissions des extensions, l'API de contrôle et l'appairage.
- Les appels aux API externes sont simulés (mocks) : les tests passent sans connexion.
- Ils se lancent en **une seule commande**.

Intégration et déploiement continus (CI/CD) obligatoire

- Intégration continue (GitHub Actions ou équivalent) déclenchée à chaque pull request : lint, tests du media center, des extensions et de la télécommande, build.
- Une pull request dont le pipeline échoue ne peut pas être mergée (vérifications obligatoires sur la branche principale).
- Déploiement continu : chaque merge sur la branche principale produit automatiquement un **paquet installable Linux** (AppImage, Flatpak ou .deb), disponible en artefact du pipeline.
- Les secrets du pipeline sont stockés dans les secrets GitHub, jamais dans le code.
- Le pipeline est décrit dans la documentation technique, et son badge d'état figure dans le README.

C. DOCUMENTATION

La documentation fait partie du produit et s'écrit au fil du projet. Elle est versionnée dans le dépôt (`/docs`, en Markdown) ou publiée depuis celui-ci (GitHub Pages, Docusaurus, MkDocs…). Langue au choix, mais la même partout.

Documentation utilisateur

Pour des personnes non techniques : elle permet d'utiliser le produit sans aide.

Public	Contenu attendu
Utilisateur	Installer le media center, ajouter ses sources, parcourir et lire ses médias, corriger une reconnaissance, installer une extension.
Télécommande	Ouvrir la télécommande sur son téléphone, l'appairer, piloter la lecture, lancer un média depuis le téléphone.
Pour tous	Guide pas à pas illustré, FAQ, dépannage (réseau, formats non lus), fonctionnalités d'accessibilité disponibles.

Guide du développeur d'extensions

Pour un développeur extérieur qui veut créer sa propre extension : API disponible, structure d'une extension, manifeste, permissions, cycle de vie, et un tutoriel qui part de zéro jusqu'à une extension simple fonctionnelle.

Documentation technique

Pour un développeur qui reprendrait le projet : elle permet de comprendre, modifier et distribuer le produit.

Rubrique	Contenu attendu
Architecture	Schéma des composants (interface, médiathèque, lecture, extensions, API, télécommande), flux de données, choix de stack justifiés.
Décisions	Journal des décisions d'architecture (ADR) : contexte, options, choix retenu, conséquences.
Modèle de données	Schéma des entités (source, média, film, série, saison, épisode, artiste, album, morceau, extension) et dictionnaire des champs.
Médiathèque	Algorithme de scan, règles de reconnaissance, stratégie d'enrichissement et de cache.
Lecture	Moteur de lecture retenu, formats supportés, sous-titres, reprise de lecture.
Extensions et thème	Architecture du système d'extensions, isolation, permissions, format du thème.
API de contrôle	Référence des commandes et événements, appairage, avec exemples.

Rubrique	Contenu attendu
Télécommande	Architecture de la page, connexion au media center, compatibilité des navigateurs mobiles.
Exploitation	Packaging, installation, mise à jour, pipeline CI/CD, résultats de l'audit d'accessibilité et des mesures de performance.

D. USAGE DE L'IA

L'usage d'outils d'IA (assistants de code, génération de contenu…) est **autorisé**. Il est encadré par les règles suivantes, conformes à la charte IA du groupe IONIS.

Transparence

Une section du README déclare les outils utilisés et pour quoi : code, tests, documentation, visuels.

Responsabilité

Le groupe est responsable de tout ce qu'il livre. Chaque membre doit pouvoir **expliquer n'importe quelle partie du code** en soutenance.

Même exigence

Le code et les contenus générés respectent les mêmes règles que le reste : tests, revue en pull request, propreté du dépôt, sécurité.

Licences

Les images, icônes et textes générés ou récupérés respectent les licences et les règles d'or. Aucune marque existante n'est imitée.

04

ORGANISATION ET LIVRABLES_

A. CALENDRIER

Toutes les activités ont lieu **en présentiel, le jeudi ou le vendredi**. Les créneaux précis sont communiqués sur My. Les quatre suivis sont **obligatoires** : chacun correspond à un jalon d'avancement.

Date	Étape	Attendu
22 octobre 2026	Lancement	Présentation du sujet en amphi. Constitution des binômes et choix du sujet sur My.
Début novembre	Suivi 1	Stack et moteur de lecture choisis, premiers ADR, dépôt, tableau GitHub Projects et CI en place.
Début décembre	Suivi 2	Médiathèque (scan, reconnaissance, enrichissement) et lecture démontrables et testées, paquet Linux produit par la CI.
Début janvier	Suivi 3	Télécommande et appairage, système d'extensions en place, charte graphique livrée.
Début février	Suivi 4	Fonctionnalités gelées, deux extensions livrées, audit d'accessibilité réalisé, documentation complète.
À partir de mi-février	Soutenance	Démo en conditions de salon pilotée au téléphone (navigation et lancement d'un média), correction d'une reconnaissance, installation d'une extension en direct, parcours au lecteur d'écran, questions sur le code pour chaque membre.
Mars	Rattrapage	Réservé aux projets non aboutis à la soutenance.

B. LIVRABLES

Livrable	Contenu
Media center	Application de salon sous Linux avec médiathèque, lecture, extensions, thème et API de contrôle.
Télécommande	Page web servie par le media center, avec appairage, pilotage et navigation dans la médiathèque.
Extensions	Les deux extensions livrées et le thème par défaut.
Paquet	Paquet installable Linux produit automatiquement par le pipeline.
Médiathèque de démo	Médias légaux avec leurs licences, et script d'installation automatique.
Dépôt GitHub	Propre, avec les fichiers indispensables, les tests, le pipeline CI/CD et le tableau GitHub Projects.
Identité du produit	Mini charte graphique (1 à 2 pages).
Documentation	Utilisateur, technique et guide du développeur d'extensions.
Accessibilité	Audit (media center et télécommande) et déclaration d'accessibilité.
Usage de l'IA	Déclaration dans le README.
Soutenance	Présentation et démo.